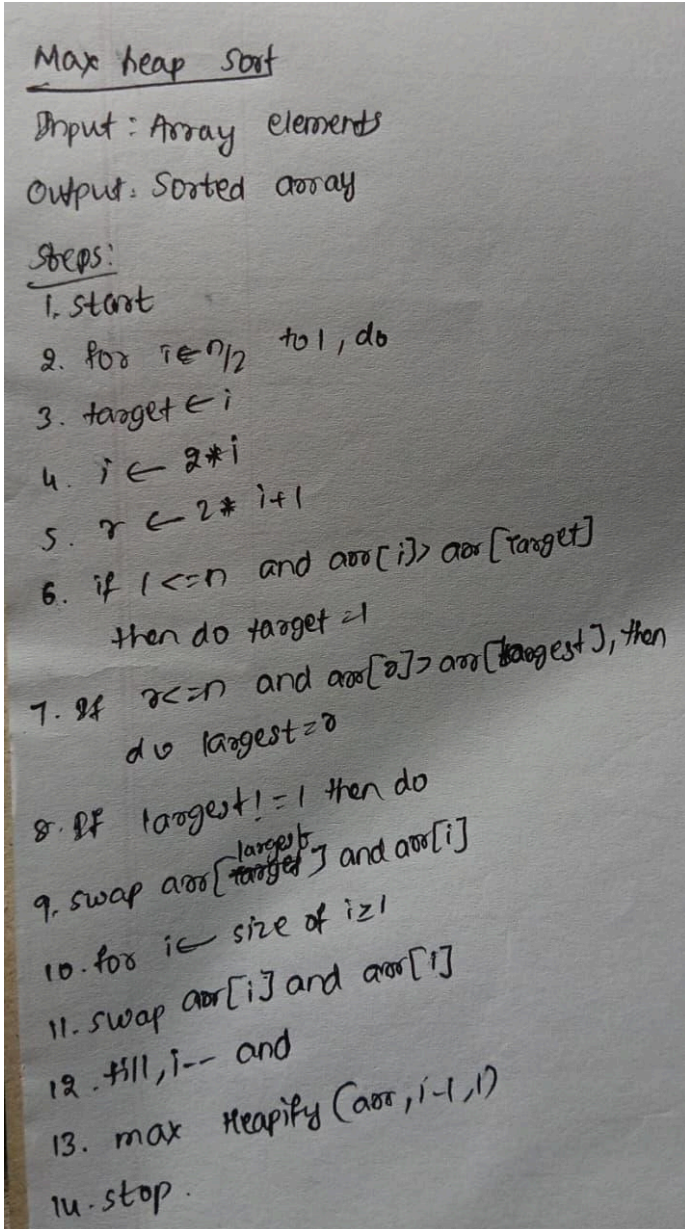


Practical-3

AIM : Implementation of max heap sort algorithm.

MAX HEAP SORT ALGORITHM:



Max heap sort
Input: Array elements
Output: Sorted array
Steps:
1. start
2. for $i \in n/2$ to 1, do
3. target $\leftarrow i$
4. $j \leftarrow 2 * i$
5. $r \leftarrow 2 * i + 1$
6. if $1 \leq n$ and $arr[i] > arr[target]$
then do target $\leftarrow i$
7. if $r \leq n$ and $arr[r] > arr[target]$, then
do largest $\leftarrow r$
8. if largest $\neq i$ then do
9. swap $arr[largest]$ and $arr[i]$
10. for $i \leftarrow$ size of $i-1$
11. swap $arr[i]$ and $arr[1]$
12. till $i--$ and
13. max Heapify ($arr, i-1, 1$)
14. stop.

CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
void heapify(int arr[], int n, int i)
{
    int largest = i;

    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);

        heapify(arr, n, largest);
    }
}
```

```
void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
```

```
heapify(arr, n, i);
```

```
// Heap Sort
```

```
for (int i = n - 1; i > 0; i--)
```

```
{
```

```
    swap(arr[0], arr[i]);
```

```
    heapify(arr, i, 0);
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
    int n;
```

```
    int arr[100];
```

```
    cout << "Enter size of array: ";
```

```
    cin >> n;
```

```
    cout << "Enter array elements: ";
```

```
    for (int i = 0; i < n; i++)
```

```
        cin >> arr[i];
```

```
    heapSort(arr, n);
```

```
cout << "Sorted array: ";
```

```
for (int i = 0; i < n; i++)
```

```
    cout << arr[i] << " ";
```

```
return 0;
```

```
}
```

OUTPUT:

```
Output
Enter size of array: 4
Enter array elements: 6 5 8 7
Sorted array: 5 6 7 8

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity (Best, Average & worst)
 for $i = \frac{n}{2}; i > 1; i-- \rightarrow n$
 maxHeapify(arr, n, i) $\rightarrow \log n$
 for $i = n; i > 1; i-- \rightarrow n$
 swap(arr[i], arr[1]) $\rightarrow 1$
 maxHeapify(arr, i-1, 1) $\rightarrow \log n$
 $T(n) = n \times \log n + n \times 1 \times \log n$
 $= n \log n + n \log n$
 $T(n) = O(n \log n)$
 for max heap sort:
 Best = Average = worst = $O(n \log n)$

CONCLUSION:

The Max-Heap Sort algorithm was successfully implemented and tested. It sorts the elements by creating a max-heap and repeatedly placing the largest element in its correct position. It is efficient for large amounts of data and has a time complexity of $O(n \log n)$.